

PULSE: Auditoria Completa (Fase 0)

Data: 2026-09-20 **Repositório:** contatoanimalzent-create/flow-events (pasta local Documents/app/flow-events) **Branch auditada:** fix/checkin-por-dia (HEAD f37c7e3) **Projeto Supabase:** nrjizzfkhficvhiiqvtl

Escopo: leitura completa de estrutura, banco em produção, migrations, componentes, páginas, Edge Functions, autenticação, RLS e funcionalidades. Nenhuma linha de código ou objeto de banco foi alterado nesta fase.

Todo número neste documento veio de execução real: `tsc` , `eslint` , `grep` sobre a árvore, e consultas SQL contra o banco de produção. Onde a evidência é indireta, está marcado como *indício*.

1. Panorama

Dimensão	Número
Arquivos <code>.ts</code> / <code>.tsx</code> em <code>src/</code>	713
Linhas de código em <code>src/</code>	40.392
Migrations versionadas	39
Edge Functions	21
Tabelas em <code>public</code> (produção)	214
Tabelas com dados reais	16
Policies RLS	194
Funções <code>public</code> expostas via RPC	49
Testes automatizados	0
Pipeline de CI	inexistente
Configuração de ESLint	inexistente

Typecheck (`tsc --noEmit`): **passa, zero erros**. É o ponto mais saudável do projeto.

2. Arquitetura atual

2.1 Stack

- **Build:** Vite 5 + TypeScript 5.2, sem SSR. SPA pura.
- **UI:** React 18, Tailwind 3, Framer Motion + GSAP + Lenis, Recharts, `react-aria-components` , `lucide-react` .
- **Estado:** Zustand 4 (stores próprios) + TanStack Query 5.
- **Backend:** Supabase (Postgres + Auth + Realtime + Storage) e 21 Edge Functions em Deno.

- **Pagamentos:** Stripe (`stripe` , `@stripe/react-stripe-js`). Não há Pagar.me implementado.
- **Mobile:** Capacitor 8 (Android + iOS), PWA com service worker e Web Push (VAPID).
- **Observabilidade:** `@sentry/react` inicializado em `src/main.tsx` .
- **Mídia:** Cloudinary + bucket Supabase.

2.2 O problema estrutural nº 1: não existe roteamento no app autenticado

`@tanstack/react-router` está no `package.json` , mas **não é usado pelo aplicativo**. Só aparece em dois arquivos órfãos (`src/shared/components/ui/AppShell.tsx` , `Layout.tsx`) que importam `Outlet` e não estão montados.

O app administrativo real vive em `src/app/layout/AppShell.tsx` e navega assim:

```
const [activeSection, setActiveSection] = useState<NavSection>(defaultNavSection)
...
<Suspense fallback=<{<PageFallback />>}>{renderSection(activeSection)}</Suspense>
```

Um `switch` sobre `useState` . Consequências diretas, todas verificáveis abrindo o sistema:

- Nenhuma tela do painel tem URL. Não dá para compartilhar link de nada.
- F5 sempre volta ao Dashboard. O usuário perde onde estava.
- O botão "voltar" do navegador sai do sistema.
- Não existe deep link para um registro (participante X, ocorrência Y).
- **Isso bloqueia diretamente** o Command Palette, a Busca Global e o Pulse AI pedidos nas Fases 1 e 10, todos precisam "navegar até um recurso", e não há endereço para onde navegar.

Ironia importante: o **app mobile (`PulseApp`) tem roteamento** via `history.pushState` (`usePulseRouter`). O painel desktop, que é o produto principal, não tem.

2.3 O problema estrutural nº 2: quatro arquiteturas de pastas convivendo

`src/` tem, ao mesmo tempo:

Pasta	O que é	Situação
<code>src/pages/</code>	26 páginas do painel admin	Viva: é o que o AppShell renderiza
<code>src/features/</code>	30 domínios (<code>checkin</code> , <code>staff</code> , <code>crm</code> ...)	Viva: as páginas são cascas finas que importam daqui
<code>src/modules/</code>	43 telas do app mobile por persona	Viva: é o <code>PulseApp</code>
<code>src/core/</code>	serviços, stores, permissões, offline, realtime	Viva: só o <code>PulseApp</code> consome
<code>src/shared/</code>	design system, hooks, utils, i18n	Viva: usada pelos dois
<code>src/components/</code>	Sidebar, Topbar, AppRouter, AppShell antigos	Legado: <code>Topbar.tsx</code> sem referência alguma
<code>src/lib/</code>	<code>supabase.ts</code> (1.753 linhas), <code>seedData.ts</code>	Misto: <code>seedData.ts</code> sem referência
<code>src/app/</code>	shell e router atuais	Viva

Existem três `AppShell.tsx` diferentes (`src/app/layout/` , `src/components/layout/` , `src/shared/components/ui/`) e dois `Sidebar` . `AppShellV2` é apenas um `export { AppShell }` , o "V2" é um alias, não uma segunda versão.

2.4 Divisão real entre os dois aplicativos

	Painel Admin ()	Pulse App (<code>/pulse</code>)
Código	<code>src/pages</code> + <code>src/features</code>	<code>src/modules</code> + <code>src/core</code>
Roteamento	❌ <code>useState</code>	✅ <code>pushState</code>
Realtime	❌ nenhum	✅ 5 hooks
Offline	❌	✅ fila + sync
Multi-org/evento	❌ sem switcher	✅ onboarding org → evento → modo
Permissões	grupos fixos na sidebar	<code>permissions.service</code> + route guard

O app mobile está arquiteturalmente **na frente** do painel. A Fase 1 deveria trazer o painel para o padrão do mobile, não o contrário.

2.5 O problema estrutural nº 3: cinco apps mobile falsos duplicando os reais

`src/features/mobile/` contém `OperatorApp` , `StaffApp` , `SupervisorApp` , `ParticipantApp` , `PromoterApp` , as **mesmas cinco personas** que `src/modules/` implementa de verdade. E as versões falsas **estão roteadas em produção** (`PublicRouteView.tsx:95-99`), acessíveis em `/operator` , `/staff-app` , `/supervisor-app` , `/participant-app` , `/promoter-app` .

`OperatorApp.tsx:334` :

```
function simulateScan() {
  const statuses: ScanStatus[] = ['valid','valid','valid','valid','invalid','duplicate']
  const names = ['Ana Lima','Carlos Melo','Julia Rosa','Pedro Silva','Maria Santos']
  const status = statuses[Math.floor(Math.random() * statuses.length)]
  const name = names[Math.floor(Math.random() * names.length)]
```

O scanner do operador nesta versão **sorteia** o resultado e o nome da pessoa. O gráfico do resumo de turno (linha 701) também é aleatório. São ~2.500 linhas de teatro roteado publicamente, ao lado de um scanner real funcional em `src/modules/operator/pages/ScannerPage.tsx` .

3. Banco de dados

3.1 O número mais importante da auditoria

214 tabelas. 16 com dados. 198 vazias.

Tabelas com dados reais em produção:

Tabela	Linhas
customers	12.090
notification_queue	3.005
order_items	2.295
digital_tickets / tickets	2.293 cada
orders	2.213
audit_logs	1.479
transactional_messages	1.015
staff_members	587
bsb_fight_ticket_claims	518
protur_educacional_inscricoes	175
status_transitions	38
checkins	37
roles / permissions	7 / 16
events	6
organizations / organization_members	4 / 5
ticket_batches / scanner_auth_codes	5 / 6

O que **não** tem uma linha sequer: incidents, gates, event_zones, venues, venue_maps, shifts, staff_shifts, teams, staff_teams, campaigns e toda a família de campanhas, sponsors, promoters, products, inventory_items, pos_*, financial_*, ai_*, operational_metrics, gate_flow_snapshots, event_health_snapshots.

Ou seja: o schema já contém a Pulse que o prompt descreve. **Só que nada dele nunca foi usado.** O sistema real hoje é: vender ingresso, emitir QR, fazer check-in, cadastrar staff.

3.2 O banco da Pulse está hospedando outros projetos

Tabelas que não pertencem ao produto e vivem no mesmo schema public:

- soberana_*, 9 tabelas de um sistema de ordens de serviço
- sentinel_leads, Sentinel Tech
- acampamento_strike_ville, capital_strike_registrations, capital_strike_qr_generation_audit_20260604
- protur_educacional_inscricoes, protur_educacional_api_config
- bsb_fight_ticket_claims, bsb_fight_ticket_api_config
- inscricoes, lista_espera, lista_espera_backup_20260604_erro_qr, lista_espera_backup_20260604_lock_final

- `vagas_exercito` (view), funções `check_limite_inscricoes_por_exercito`, `block_lista_espera_insert_capital_strike`

Isso é o oposto exato do requisito de multi-tenancy. Cada um desses clientes deveria ser uma `organization` dentro do modelo da Pulse, não um conjunto de tabelas cravadas no schema com regras próprias. Enquanto estiverem aí, "nenhum dado pode vazar entre organizações" é uma afirmação que não se sustenta, e várias dessas tabelas são justamente as que estão com RLS quebrada (ver 3.5).

3.3 Multi-tenancy: um usuário, uma organização

As duas funções que sustentam toda a RLS:

```
-- auth_org_id()      usada por 46 policies
SELECT organization_id FROM profiles WHERE id = auth.uid() LIMIT 1;

-- get_user_org_id()  usada por 53 policies
SELECT organization_id FROM public.profiles WHERE id = auth.uid();
```

Três problemas:

1. **São a mesma função, duplicada.** Comportamento idêntico, nomes diferentes, 99 policies divididas entre as duas. Qualquer correção futura precisa ser feita em dois lugares ou o modelo diverge.
2. **A organização do usuário é um campo escalar em `profiles`.** Um usuário pertence a exatamente uma org. O requisito é "um usuário pode participar de múltiplas organizações e eventos".
3. **A tabela `organization_members` existe, tem 5 linhas, e a RLS a ignora.** O modelo N:N está modelado, populado e não é consultado por nenhuma das 99 policies. Só duas policies de `staff_arrival_proofs` usam `organization_members`.

Há ainda 1 perfil com `organization_id` nulo dos 5 existentes. Para esse usuário, toda policy org-scoped resolve `= NULL` → falso → ele não enxerga nada e não recebe erro explicando o porquê.

3.4 RBAC: dois modelos concorrentes, nenhum autoritativo

No banco: `roles` (7), `permissions` (16), `role_permissions`, `member_permission_overrides`. Tabelas populadas.

No código (`src/core/permissions/permissions.service.ts:5`):

```
const ROLE_MODE_MAP: Record<string, AppMode[]> = {
  super_admin: ['operator', 'staff', 'supervisor', 'attendee', 'promoter'],
  org_admin:   ['operator', 'staff', 'supervisor', 'attendee', 'promoter'],
  ...
}

const MODE_MODULES: Record<AppMode, PermissionModule[]> = { ... }
```

As permissões efetivas são calculadas **no cliente**, a partir de um mapa hardcoded em TypeScript. As tabelas `roles` / `permissions` / `role_permissions` **não são lidas por esse serviço**. O que o banco entende por permissão e o que o frontend entende por permissão são dois sistemas independentes.

Além disso, o serviço identifica o staff do usuário por **string de e-mail**, não por FK:

```
supabase.from('staff_members').select('role_title').eq('email', userEmail).eq('event_id', eventId)
```

Trocar o e-mail de um membro quebra silenciosamente a permissão dele.

3.5 Vazamento entre organizações: a falha mais grave

35 policies usam USING (true) e **8 usam auth.role() = 'authenticated'** . Em ambos os casos, qualquer usuário logado de qualquer organização lê a tabela inteira.

Tabelas afetadas, com o que exatamente vaza:

Tabela	Policy	O que qualquer usuário logado enxerga
staff_checkins	staff_checkins_public_select → true	Todos os check-ins de equipe, de todos os eventos e orgs
commissions	commissions_select → true / commissions_all → authenticated	Comissões financeiras de todos os promoters de todas as orgs
promoter_ranking , promoter_goals	true + authenticated	Performance comercial de terceiros
referral_links , referral_conversions , referral_clicks	true + authenticated	Funil de aquisição de qualquer org
supervisor_approvals	true + authenticated	Aprovações operacionais de qualquer evento
team_invites	true + authenticated	Convites de equipe, com tokens
staff_teams , staff_instructions	true	Estrutura de equipes e briefings de qualquer evento
venue_maps	venue_maps_read → true	Plantas operacionais de qualquer local
protur_educacional_inscricoes	autenticados_podem_ler → true	175 inscrições reais com PII
soberana_* (8 tabelas)	*_staff_all → true	CRUD completo, por qualquer logado, em dados de outro cliente
platform_fee_rules	auth.uid() IS NOT NULL	Regras de taxa da plataforma
agenda_sessions , agenda_favorites , feed_likes	true	menor impacto

Isto não é hipótese: é o predicado literal das policies em produção. `commissions` e `soberana_*` são os piores casos, dados financeiros e de cliente terceiro com leitura (e no caso soberana, escrita) irrestrita.

3.6 RLS desligada ou sem policy

RLS desabilitada (leitura anônima via API REST): - `staff_checkin_offline_log`, **tabela operacional real** - `capital_strike_qr_generation_audit_20260604` - `lista_espera_backup_20260604_erro_qr` - `lista_espera_backup_20260604_lock_final` - `spatial_ref_sys` (PostGIS, aceitável)

RLS ligada e zero policies (efeito colateral: a tabela fica inacessível pelo cliente, só `service_role` funciona: quebra silenciosa, não é proteção intencional): `acampamento_strike_ville`, `bsb_fight_ticket_api_config`, `bsb_fight_ticket_claims` (518 linhas), `lista_espera`, `protur_educacional_api_config`, `sentinel_leads`.

3.7 Superfície RPC exposta

49 funções SECURITY DEFINER são executáveis pelo role anon via `/rest/v1/rpc/<nome>`, incluindo `auth_org_id()`, `check_rate_limit()`, `confirm_order_and_capture_inventory()`, `create_order_draft_with_reservations()`. 47 são executáveis por `authenticated`, incluindo `create_organization()`.

`SECURITY DEFINER` roda com os privilégios do dono, ignorando RLS. Cada uma dessas 49 precisa ter a autorização verificada no próprio corpo da função, caso contrário é um bypass de RLS acessível sem login. Não auditei o corpo das 49; é item obrigatório da Fase 1.

Adicionalmente: **6 views SECURITY DEFINER** (`staff_attendance_live`, `staff_hours_today`, `staff_event_summary`, `bsb5_daily_report`, `bsb5_summary_by_day`, `vagas_exercito`), views assim ignoram a RLS das tabelas de base para quem as consulta. **16 funções com search_path mutável** (vetor clássico de escalada de privilégio).

3.8 Duplicação de modelo

Pares e trios que representam o mesmo conceito:

- `staff` (0 linhas) × `staff_members` (587), `staff` é morta
- `checkins` (37) × `checkin_logs` × `checkin_attempts` × `qr_scan_attempts`
- `tickets` (2.293) × `digital_tickets` (2.293), espelhamento 1:1
- `customers` (12.090) × `contacts` × `profiles` × `person_event_profiles` × `customer_event_profiles`
- `coupons` × `discount_coupons`
- `time_entries` × `timeclock_entries` × `timeclock_sessions` × `staff_presence_sessions` × `staff_checkins`
- `event_zones` × `venue_zones`
- `shifts` × `staff_shifts` | `teams` × `staff_teams`
- `notification_jobs` × `notification_queue` × `internal_notifications`
- `ai_alerts` × `ai_suggestions` × `ai_recommendations` × `operational_alerts` × `intelligence_alert_states`
- `gates` × `event_access_points` × `checkpoints`
- `webhooks` × `webhook_logs` × `webhook_deliveries`

3.9 Performance no banco

Achado	Contagem
FKs sem índice de cobertura	259
Índices nunca utilizados	298
Policies re-avaliando <code>auth.*()</code> por linha (<code>auth_rls_initplan</code>)	35
Múltiplas policies permissivas para mesma role+ação	270
Índices duplicados	1 (<code>person_event_profiles</code>)
Tabelas sem primary key	3 (todas de backup)
Extensão <code>postgis</code> no schema <code>public</code>	1

259 FKs sem índice + 298 índices inúteis é o retrato de um schema criado por migrations sucessivas sem revisão de carga. Os 35 `auth_rls_initplan` são correção mecânica e de alto retorno (`auth.uid()` → `(SELECT auth.uid())`).

4. Frontend

4.1 Navegação: 26 seções

`src/app/layout/navigation.ts` declara 26 `NavSection` , agrupadas em 5 grupos. O prompt pede 10 áreas (HOME, PLAN, SELL, ACCESS, WORKFORCE, ENGAGE, COMMAND, INTELLIGENCE, REPORTS, SETTINGS).

Mapeamento do que existe hoje para o alvo:

Alvo	Seções atuais que colapsam nele
HOME	<code>dashboard</code>
PLAN	<code>events</code>
SELL	<code>tickets</code> , <code>sales</code> , <code>products</code> , <code>inventory</code> , <code>coupons</code> , <code>waitlist</code> , <code>financial</code> , <code>billing</code> , <code>monetization</code> , <code>growth</code>
ACCESS	<code>registrations</code> , <code>checkin</code>
WORKFORCE	<code>staff</code> , <code>suppliers</code>
ENGAGE	<code>crm</code> , <code>communication</code> , <code>community</code> , <code>sponsors</code>
COMMAND	<code>map</code> + (não existe)
INTELLIGENCE	<code>intelligence</code> , <code>audit</code>
REPORTS	não existe
SETTINGS	<code>settings</code> , <code>organizations</code> , <code>help</code>

Além disso, `getSectionMeta()` mantém um segundo catálogo de 26 itens "ocultos" com labels diferentes dos visíveis, duplicação de metadados que já divergiu (ex.: `intelligence` é "IA Operacional" na sidebar e "Inteligência" no catálogo oculto; `suppliers` usa ícone `Building2` visível e `CircleHelp` oculto).

4.2 O que existe e funciona

✅ **Funcionando com dados reais:** - Login / signup / troca de senha obrigatória / gate de organização - Catálogo público de eventos, landing de evento, checkout Stripe - Emissão de ingresso digital + QR (`digital_tickets`, 2.293 linhas) - Check-in por QR com validação server-side (Edge Function `validate-checkin`), suporte a offline e a check-in por dia em evento multi-dia - Kiosk de autoatendimento e scanner de operador (`ScannerPage`, `Scanner2Page`) - Cadastro e convite de staff, links de convite, `StaffJoinPage`, `StaffTimeclockPage`, ponto - CRM de clientes (12.090 registros) - Fila de notificações transacionais (e-mail via Resend, WhatsApp via Twilio não configurado) - Audit log gravando (1.479 registros) - Sentry ativo - App mobile Pulse com multi-org, multi-evento, multi-role, offline e realtime - Integrações verticais reais: BSB Fight, Protur Educacional, Capital Strike, Nocaute

4.3 O que está parcial

⚠️ - **Dashboard**, mistura métricas reais com um gráfico fabricado (ver 4.5) - **Mapa Operacional**, existe a tela e o builder de planta, mas `event_zones`, `gates`, `venue_maps` estão vazios; não há fonte de dados viva - **Intelligence**, `intelligence.calculations.ts` (548 linhas) é real, mas alimenta-se de tabelas com zero linhas - **Pulse AI** (`ai-operational-assistant`), Edge Function existe, sem separação READ / SUGGEST / EXECUTE e sem confirmação para ação sensível - **Financeiro / Billing / Monetização / Growth / Community / Sponsors**, telas completas sobre tabelas vazias - **Offline**, fila implementada (`offline.sync.ts`), consumida só pelo app mobile; sem resolução de conflito explícita nem indicador ONLINE/OFFLINE/SINCRONIZANDO padronizado

4.4 O que não existe

❌ - **Command Center** como área própria. Existe `CommandCenterOverview.tsx`, mas dentro de `features/checkin`, é um painel de check-in, não a central operacional - **Incident Management**. A tabela `incidents` existe e está vazia; **não há nenhuma tela, serviço ou hook** que a leia ou escreva. A palavra só aparece em páginas de marketing. O módulo inteiro está por fazer - **Busca global**, nenhuma implementação - **Command Palette (Cmd/Ctrl+K)**, só em `src/components/layout/Topbar.tsx`, arquivo sem nenhuma referência (morto) - **Event/Org Switcher no painel admin**, o `Header` não tem. Só o app mobile tem - **Notification Center no admin**, existe em `modules/shared-shell` (mobile), não no painel - **Reports**, não há área de relatórios - **Networking, Sponsors Portal, Lead Capture, Pulse ID, Marketplace, Autopilot**, Fases 7, 8, 11, 12, 13, nada iniciado - **Onboarding de organização**, não existe no painel

4.5 Dados inventados apresentados como reais

`src/components/dashboard/RecentOrdersCard.tsx:115-125` :

```

setData(chartData.length > 0 ? chartData : generateMockData())
...
function generateMockData(): HourlyData[] {
  return Array.from({ length: 12 }, (_, i) => ({
    hour: `${8 + i}h`,
    checkins: Math.floor(Math.random() * 120),
  })))
}

```

Quando o evento não tem check-in, o card **desenha 12 horas de movimento inventado**, e o cabeçalho ao lado exibe um ponto verde pulsante com a legenda **"Ao vivo"**. É a violação mais direta da regra "gráfico com dado inventado".

`src/features/mobile/operator/OperatorApp.tsx:334 e 701` : scanner sorteado e gráfico de turno aleatório, descrito em 2.5.

`src/features/mobile/participant/ParticipantApp.tsx:504` : "Pagamento via Pagar.me, integração em breve". A Pulse é 100% Stripe; Pagar.me nunca existiu.

4.6 UX

- **Sem URL** → sem link compartilhável, sem voltar, sem F5 (item 2.2)
- **26 itens de menu** contra as 10 áreas pedidas
- **Nenhum Event Switcher no admin**, trocar de evento exige navegar até Eventos
- **83** `.select('*')` e **zero** `.range()`, não há paginação em lugar nenhum. `customers` tem 12.090 linhas: a tela de CRM tenta trazer tudo para o cliente
- **86 arquivos de UI chamam** `@/lib/supabase` **diretamente**, a camada de serviço existe (`src/features/*/services`) mas é contornada
- `src/lib/supabase.ts` **com 1.753 linhas**, arquivo-monólito no caminho crítico de praticamente toda tela
- **Zero states fracos**, "Comunidade será ativada em breve", "Ativações de marca serão configuradas em breve" são avisos, não caminhos de ação
- **Cinco telas mobile falsas roteadas em produção** ao lado das reais, um usuário que abrir `/operator` usa um simulador

4.7 Qualidade e processo

- `npm run lint` **está quebrado desde sempre**. Não existe `.eslintrc*` nem `eslint.config.js`. O comando aborta com "ESLint couldn't find a configuration file". Nunca houve análise estática.
- **Zero testes**. Nenhum `*.test.*`, `*.spec.*`, `vitest.config`, `playwright.config` em `src/`. (Há testes em `aios-core/`, que é ferramental, não o produto.)
- **Sem CI**. Não existe `.github/workflows/`.
- **Sem Biome, Knip, commitlint, dependency-cruiser, Codecov**, nada do padrão de produção definido no CLAUDE.md global.
- `docs/` tem `ARCHITECTURE_V2.md`, `FOLDER_STRUCTURE.md`, `ROADMAP.md` e nenhum deles reflete o estado atual (falam de uma arquitetura V2 que é um alias).

5. Dívida técnica consolidada

#	Item	Impacto
D1	Painel admin sem roteamento por URL	Bloqueia Fases 1, 5, 10
D2	Quatro arquiteturas de pasta, três <code>AppShell</code> , dois <code>Sidebar</code>	Custo de toda mudança
D3	5 apps mobile falsos roteados em produção	Credibilidade + confusão
D4	198 de 214 tabelas vazias	Schema que não corresponde ao produto
D5	Outros 6 projetos hospedados no schema <code>public</code> da Pulse	Viola multi-tenancy
D6	<code>auth_org_id() = get_user_org_id()</code> duplicadas em 99 policies	Risco de divergência
D7	Um usuário = uma organização; <code>organization_members</code> ignorada	Viola requisito de negócio
D8	RBAC hardcoded no cliente × tabelas <code>roles</code> / <code>permissions</code> no banco	Segurança no frontend
D9	43 policies com leitura irrestrita cross-org	Vazamento de dados
D10	49 RPCs <code>SECURITY DEFINER</code> abertas ao <code>anon</code>	Possível bypass de RLS
D11	5 tabelas com RLS desligada, 6 com RLS sem policy	Exposição / quebra silenciosa
D12	259 FKs sem índice, 298 índices mortos, 35 <code>initplan</code> , 270 policies sobrepostas	Performance
D13	<code>supabase.ts</code> com 1.753 linhas; 86 componentes acessando o cliente direto	Manutenção
D14	Zero paginação (83 <code>select('*', 0 .range())</code>)	Escala
D15	ESLint sem config, zero testes, sem CI	Nenhuma rede de proteção
D16	~13 pares/trios de tabelas duplicadas para o mesmo conceito	Fonte da verdade ambígua
D17	Dados fabricados rotulados "Ao vivo"	Viola regra explícita do produto
D18	Realtime existe e o painel admin não usa	Contradiz "tempo real"

6. O que deve ser preservado

Nada disto deve ser reescrito. É a base real da Pulse:

- Fluxo de venda** → **ingresso** → **QR** → **check-in**. 2.213 pedidos, 2.293 ingressos, 37 check-ins. Funciona.
- Edge Functions de check-in e emissão** (`validate-checkin`, `operator-ticket-checkin`, `generate-credential`, `staff-checkin`, `scanner-auth`), validação server-side já está no lugar certo.
- Integração Stripe completa** (`stripe-checkout`, `stripe-payment-intent`, `stripe-webhook`, `payment_webhook_events`).
- `src/modules/` + `src/core/`, o app mobile é a melhor arquitetura do repositório. Roteamento, offline, realtime, multi-org, route guard. **Deve virar o padrão do painel.**

5. **Cadastro e operação de staff**, 587 `staff_members` reais, convites, ponto, geofence (`event_geofences` , `staff_arrival_proofs` , `compute_arrival_proof_distance`).
 6. **CRM**, 12.090 clientes.
 7. `audit_logs` , já grava, 1.479 registros. Estender, não recriar.
 8. `intelligence.calculations.ts` , 548 linhas de cálculo legítimo esperando dados.
 9. **Design system** em `src/shared/design-system` e os tokens `--pulse-*` .
 10. **Sentry, PWA, Web Push, Capacitor**, infraestrutura pronta.
 11. **Integrações verticais** (BSB Fight, Protur, Capital Strike, Nocaute), geram receita hoje. Devem ser **migradas** para o modelo de organização, nunca apagadas.
-

7. Prioridades críticas

P0: parar o sangramento (fazer antes de qualquer feature):

1. Fechar as 43 policies com leitura cross-org (`commissions` , `staff_checkins` , `soberana_*` , `referral_*` , `promoter_*` , `supervisor_approvals` , `team_invites` , `protur_educacional_inscricoes` , `platform_fee_rules`).
2. Auditar o corpo das 49 RPCs `SECURITY DEFINER` expostas ao `anon` e revogar o que não precisar de acesso anônimo.
3. Ligar RLS em `staff_checkin_offline_log` ; criar policy ou revogar grants nas 6 tabelas com RLS sem policy; arquivar as 3 tabelas de backup.
4. Remover das rotas os 5 apps mobile falsos e o `generateMockData()` do Dashboard.
5. Corrigir as 6 views `SECURITY DEFINER` e os 16 `search_path` mutáveis.

P1: fundação:

1. Roteamento por URL no painel admin.
2. Unificar `auth_org_id` / `get_user_org_id` em uma única função baseada em `organization_members` (multi-org).
3. Mover o RBAC para o banco (`roles` / `permissions` / `role_permissions`) e ler no servidor.
4. ESLint + CI + primeiros testes de caminho crítico.
5. Event/Org Switcher + Busca Global + Command Palette.

P2: consolidação:

1. Colapsar 26 seções em 10 áreas.
 2. Unificar `src/pages` + `src/features` + `src/modules` num padrão só.
 3. Extrair as verticais (BSB, Protur, Capital Strike, Soberana, Sentinel) para organizações.
 4. Paginação, índices de FK, limpeza de índices mortos.
 5. Podar as tabelas vazias que não têm dono.
-

8. Plano detalhado da Fase 1

Objetivo declarado: *"fazer a Pulse ser extremamente estável"*. Traduzido para este repositório, estabilidade significa **fechar o vazamento, dar endereço às telas e tornar as permissões verificáveis no servidor**. Nesta ordem.

Sprint 1.0: Contenção de segurança (sem mudança de UI)

Entrega: migration `20260921_rls_cross_tenant_lockdown.sql`, reversível.

- Substituir todas as policies `USING (true)` e `auth.role()='authenticated'` por predicado org-scoped, tabela a tabela, com a lista da seção 3.5.
- `ALTER TABLE staff_checkin_offline_log ENABLE ROW LEVEL SECURITY` + policy.
- Policy explícita ou `REVOKE` nas 6 tabelas com RLS sem policy.
- Mover as 3 tabelas de backup para schema `archive`.
- `SET search_path = public, pg_temp` nas 16 funções.
- Converter as 6 views para `security_invoker = true` (validando que os consumidores continuam funcionando).
- Revisar as 49 RPCs: manter `anon` apenas onde o fluxo público exige (`claim_bsb_fight_ticket_public`, `claim_protur_ticket_public`, `checkout`), `REVOKE EXECUTE FROM anon` no resto.

Verificação: para cada tabela tocada, executar `set local role authenticated; set request.jwt.claims` com um usuário da org A e confirmar zero linhas da org B. Sem isso o sprint não fecha.

Sprint 1.1: Multi-tenancy verdadeiro

- Nova função única `pulse_current_org_ids()` retornando `setof uuid` a partir de `organization_members`.
- Policies migram de `= auth_org_id()` para `IN (SELECT pulse_current_org_ids())`.
- Backfill: para cada `profiles.organization_id` não nulo, garantir linha em `organization_members`.
- `auth_org_id()` e `get_user_org_id()` viram wrappers deprecados (não removidas na mesma migration).
- Resolver o perfil com `organization_id` nulo.
- Aplicar `(SELECT auth.uid())` nas 35 policies com `auth_rls_initplan`.
- Criar índices nas FKs de `organization_id` e `event_id` das tabelas com dados.

Sprint 1.2: RBAC no servidor

- Popular `role_permissions` com o conteúdo de `ROLE_MODE_MAP` + `MODE_MODULES`.
- Função `pulse_effective_permissions(p_event_id uuid)` resolvendo perfil → org → role → permissions + `member_permission_overrides`.
- `permissions.service.ts` passa a chamar essa função. O mapa hardcoded é deletado.
- Trocar o match por e-mail em `staff_members` por FK `user_id` (com backfill por e-mail e coluna mantida durante a transição).
- `route-guard.ts` passa a consumir o resultado do servidor.

Sprint 1.3: Roteamento real no painel

- Adotar no `AppShell` o mesmo `usePulseRouter` (`pushState`) que o app mobile já usa, não introduzir uma terceira biblioteca.
- Rotas: `/app/:org/:event/:section` e `/app/:org/:event/:section/:id`.
- `AppShell` deixa de ser `switch(useState)` e passa a ler a rota.
- Remover `@tanstack/react-router` do `package.json` e os dois arquivos órfãos que o importam.
- Preservar todas as rotas públicas atuais (`public-routes.ts` continua no lugar).

Sprint 1.4: Contexto global e busca

- **Event/Org Switcher** no `AppHeader`, lendo `organization_members` + `events`, reaproveitando os stores de `src/core/organizations` e `src/core/events`.
- **Busca Global:** RPC única `pulse_global_search(q text, p_org uuid, p_event uuid)` com `UNION ALL` sobre `customers`, `staff_members`, `digital_tickets`, `events`, `orders`, `suppliers`, `server-side`, limitada, org-scoped. Nunca buscar no cliente.
- **Command Palette Cmd/Ctrl+K** sobre a busca global + ações ("Criar evento", "Adicionar staff", "Trocar evento"). Deletar `src/components/layout/Topbar.tsx`.

Sprint 1.5: Rede de proteção

- `eslint.config.js` (flat config) com `typescript-eslint` + `react-hooks`; corrigir o passivo até `--max-warnings 0`.
- Vitest para `permission.utils`, `checkin.service`, `intelligence.calculations`.
- Playwright no caminho crítico: login → seleciona org → seleciona evento → dashboard → check-in.
- GitHub Actions: `tsc` + `lint` + `test` + `build` em todo PR.
- Knip para confirmar o código morto antes de remover (`Topbar.tsx`, `seedData.ts`, `src/shared/components/ui/AppShell.tsx`, `Layout.tsx`).

Sprint 1.6: Limpeza visível

- Remover as rotas `/operator`, `/staff-app`, `/supervisor-app`, `/participant-app`, `/promoter-app` e deletar `src/features/mobile/*` (as reais ficam em `src/modules/*` sob `/pulse`).
- Remover `generateMockData()` de `RecentOrdersCard` e substituir por empty state com ação.
- Remover a menção a Pagar.me.
- Unificar os três `AppShell` e dois `Sidebar` em um de cada.

Fora do escopo da Fase 1 (registrado para não se perder)

- Colapso de 26 → 10 seções (entra na Fase 2, junto com o novo fluxo de criação de evento).
- Extração das verticais para organizações (Fase 2/3, precisa de janela combinada com cada cliente).
- Poda das 198 tabelas vazias (só depois que as Fases 3-9 definirem quais serão usadas).

Definition of Done da Fase 1

Uma fase só fecha com: migrations reversíveis aplicadas e verificadas com consulta cross-org real; `tsc` + `lint` + `test` + `build` verdes; login → org → evento → dashboard funcionando em desktop e mobile; nenhuma

policy USING (true) restante em tabela com dado de cliente; e o relatório no formato pedido (implementado / migrations / arquivos / testes / segurança / UX / pendências / próxima fase).

9. Riscos da execução

Risco	Mitigação
Fechar RLS quebra telas que dependiam da leitura aberta	Fazer tabela a tabela, com verificação cross-org, em migrations pequenas e reversíveis
REVOKE nas RPCs derruba fluxo público de BSB Fight / Protur (receita ativa)	Auditar cada RPC antes; manter explicitamente as de claim público
Migrar para organization_members com profiles.organization_id ainda em uso	Backfill primeiro, wrappers deprecados, remoção só depois de duas fases
Introduzir roteamento quebra estado das telas	Reaproveitar o usePulseRouter já provado no app mobile
Sem testes, qualquer refactor é às cegas	Sprint 1.5 antes das fases de produto; Playwright no caminho crítico primeiro

Fase 0 concluída. Nenhuma alteração de código ou banco foi realizada.